Beyond coverage tracks: fine-tuning AlphaGenome's splicing heads from scratch

Miquel Anglada-Girotto, Jonathan Frazer, Mafalda Dias

Center for Genomic Regulation (CRG)

Genomics x AI Blog, 24 July 2026. https://doi.org/10.5281/zenodo.21535145
Online version: https://genomicsxai.github.io/blogs/2026-009/

Summary

DeepMind has released AlphaGenome's code and model weights, and the community has since developed alphagenome_ft and alphagenome-pytorch to enable seamless fine-tuning in both JAX and PyTorch. These implementations support bigwig-based modalities such as RNA sequencing (RNA-seq) coverage tracks, but other RNA-seq-derived outputs, including splice site probabilities, splice site usage, and splice junctions, require additional work, from data preprocessing and loading to validating fine-tuning on samples not represented during AlphaGenome pretraining. What initially seemed like a straightforward extension became a useful exercise in understanding how large genomic models learn and how to debug new output heads.

In this post, we share that development process: preprocessing the data, building loaders, running sanity checks, and overfitting a single interval, including the bugs we found along the way and how we fixed them. We then scale up to full fine-tuning and evaluation on held-out genomic intervals and report the resulting performance for two fine-tuning strategies.

All code, model adaptations, and pipelines used for this blog post are available.

Motivation

This post describes how we fine-tuned and evaluated AlphaGenome's splicing heads, covering data preprocessing, loading, debugging, and testing on held-out genomic intervals. Our broader goal is to develop a workflow that can adapt AlphaGenome to new RNA-seq samples that were not represented during the model's original pretraining. The workflow learns sample-specific, multimodal transcriptomic patterns from those data and predicts them at genomic intervals held out during fine-tuning.

As a case study, we selected two RNA-seq samples from López-Oreja et al. (2023): one carrying the cancer-driver mutation K700E in SF3B1, a core component of the spliceosome, and one wild-type sample. Although K700E changes only a single amino-acid residue, it can affect splicing across the transcriptome. In particular, the mutation is known to promote the use of cryptic splice sites, making it a useful test case for the RNA-seq-derived modalities considered here.

What comes from where

Since much of this work sits between “using AlphaGenome as published,” “reproducing details the paper doesn’t fully specify,” and “our own implementation choices,” here’s a breakdown of which parts of the pipeline fall into each category:

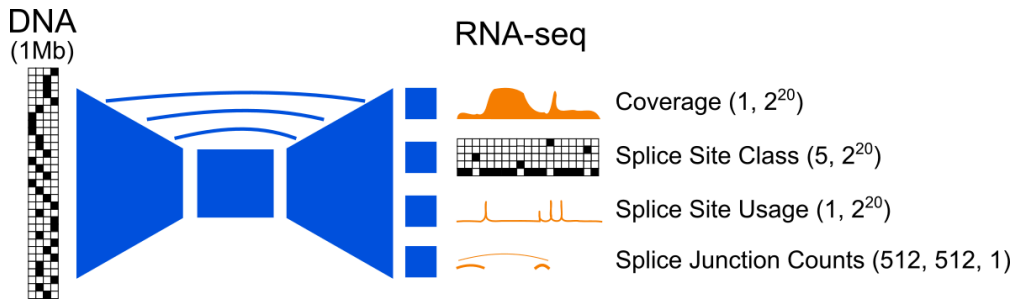
Component	Category
Junction-head architecture and candidate splice-site conditioning	Inherited from AlphaGenome, unmodified
STAR/deepTools-based alignment and coverage processing	Reproduced from the AlphaGenome paper’s methods, best-effort (original preprocessing code is not public)
Splice site usage / junction count extraction scripts, on-the-fly per-modality normalization	New implementation choices, specific to this project
`rope_params` truncated-normal initialization, ratio-normalized junction loss	Debugging fixes for a mismatch between the paper’s pseudocode and the JAX/PyTorch implementations — not new methodological contributions

AlphaGenome’s splicing heads

AlphaGenome predicts four RNA-seq-derived modalities: coverage, splice site class, splice site usage, and splice junction counts.

The coverage head predicts read depth along the sequence. The splice site head classifies each position as donor, acceptor, or background for each strand. The splice site usage head predicts how strongly each splice site is used, and the junction head predicts counts for candidate donor–acceptor pairs.

Coverage, splice site usage, and junction counts use sample-specific output tracks, whereas the splice site head is shared across samples. The junction head also receives candidate donor and acceptor positions as part of its input and predicts a normalized count for each candidate pair.

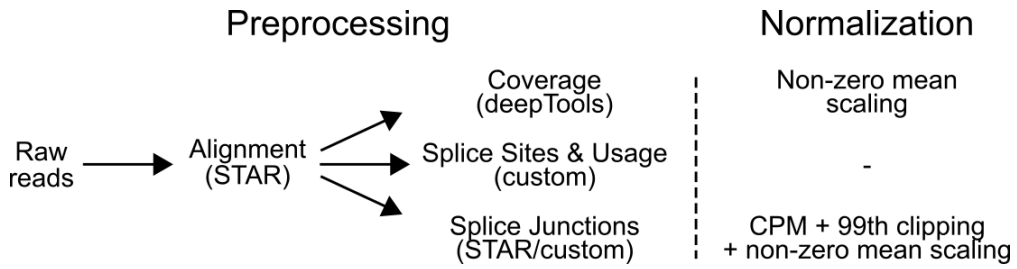


Overview of AlphaGenome’s splicing heads.

From raw data to preprocessed training tracks

AlphaGenome’s data preprocessing code is not publicly available, though the key steps are described in the paper’s methods. To reproduce the pipeline as closely as possible, we wrote standardized Snakemake workflows that derive all four training tracks from a single STAR RNA-seq alignment BAM file. Reads are aligned with STAR following the AlphaGenome paper’s alignment settings, retaining only uniquely mapped reads on canonical chromosomes.

- **Per-base RNA-seq coverage** (stranded or unstranded) is computed using `deepTools`' `bamCoverage`. Output: bigwig files.
- **Splice site classes** are derived on the fly during data loading from the union of splice sites present in the splice site usage files, so no separate file is needed.
- **Splice site usage** is computed from the BAM using our custom script `compute_ssu.py`, equivalent to `SpliSER`. For each splice site supported by at least one junction, usage is estimated as the fraction of reads supporting that site relative to reads that skip it. Output: zstd-compressed parquet files.
- **Splice junction counts** are available directly from STAR at alignment time, or can be extracted post-hoc from the BAM using our custom script `get_star_junctions.py`. Output: tab-separated files.

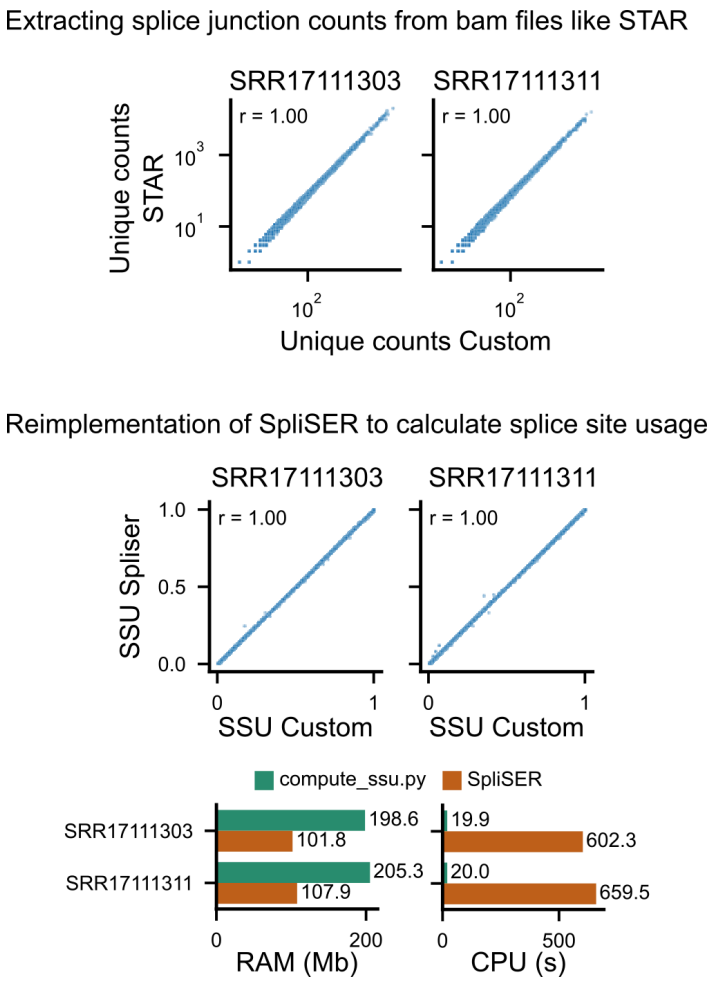


Data preprocessing workflow.

To make sure our custom implementations to compute splice junction counts and splice site usage matched how STAR and SpliSER calculate them respectively, we ran a comparison on chromosome 1 on our samples.

In both cases we observe a Pearson correlation of 1 (for splice junction counts from uniquely mapped reads and for splice site usage), confirming the implementations are correct.

Both implementations were also highly efficient in runtime, with `get_star_junctions.py` requiring ~10s and ~150 MB per sample, and `compute_ssu.py` requiring ~20s and ~200 MB per sample compared to SpliSER's ~600s and ~100 MB.



Benchmark of splice junction counting and splice site usage computation.

Data loading and on-the-fly normalization

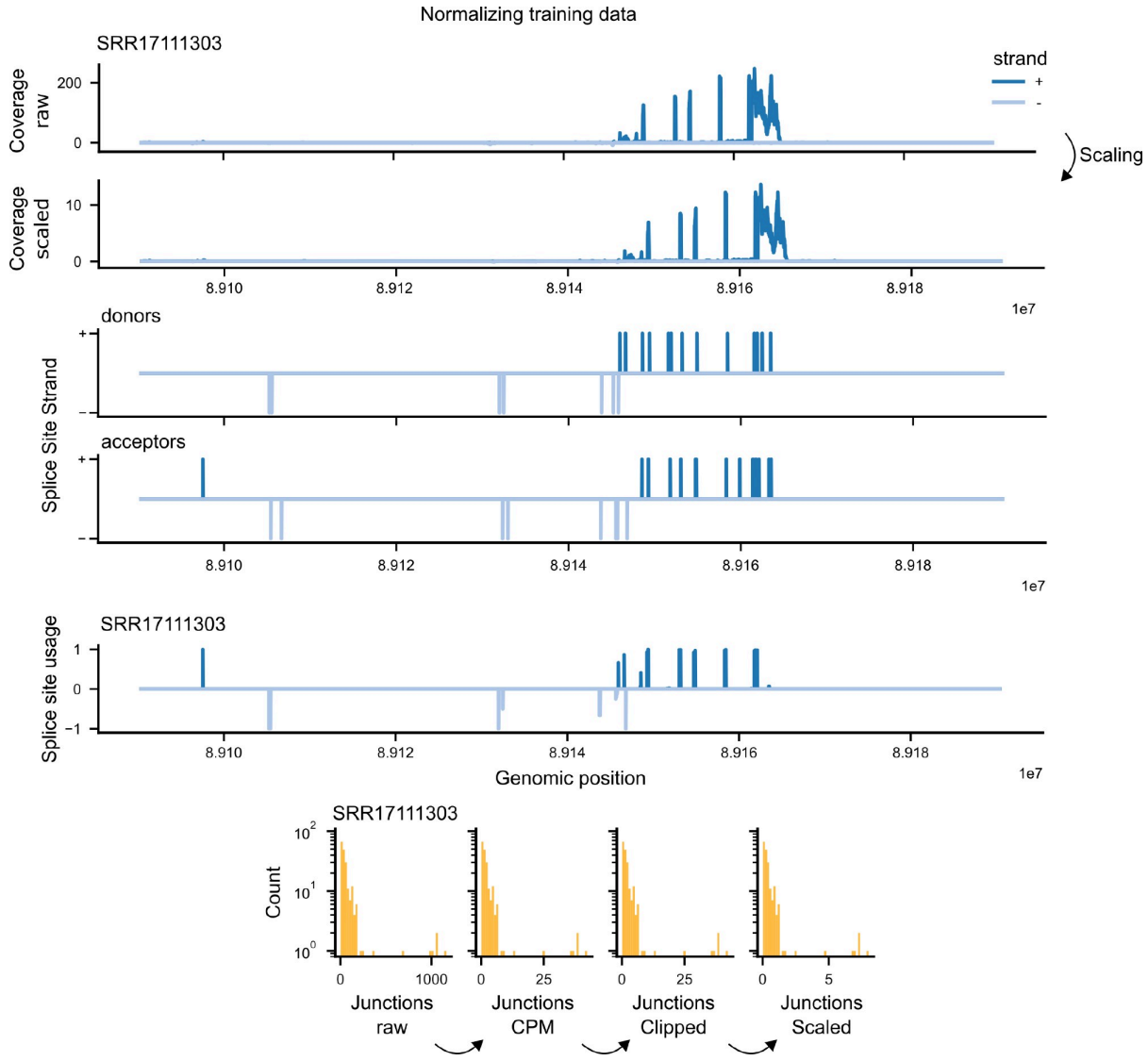
All four tracks are loaded jointly for each genomic interval and normalized on the fly before being passed to the model. Getting this right matters: RNA-seq coverage spans several orders of magnitude across genes and samples, splice junction counts are extremely sparse, and splice site labels are binary signals at a handful of positions in a million-base-pair window. Each modality needs a different normalization strategy to avoid one dominating the loss or presenting the model with uninformative targets.

RNA-seq coverage bigwigs are read at 1 bp resolution. The raw values are loaded as-is; normalization happens at training time. Just before the loss is computed, targets are scaled into model prediction space by dividing by a per-track non-zero mean (computed once from the training set), applying a power transform (x^0.75) to compress dynamic range, and smooth-clipping extreme values. Model predictions are converted back to data space using the inverse of these operations for evaluation.

Splice site classes are derived on the fly as a 5-class label array: donor on the positive strand, acceptor on the positive strand, donor on the negative strand, acceptor on the negative strand, and background. All positions default to background, and annotated splice sites from the union of sites in the splice site usage files are then overlaid.

Splice site usage values are loaded from the precomputed parquet files. For each genomic interval, only the splice sites overlapping that interval are retrieved, and usage values are arranged into a per-position array with two channels per sample (one per strand).

Splice junction counts from STAR are CPM-normalized using the total mapped reads per sample, clipped at the 99.99th percentile, and then mean-scaled so that the typical non-zero value is close to 1. Junctions are assembled into a donor × acceptor count matrix, with forward and reverse strand channels interleaved across samples.

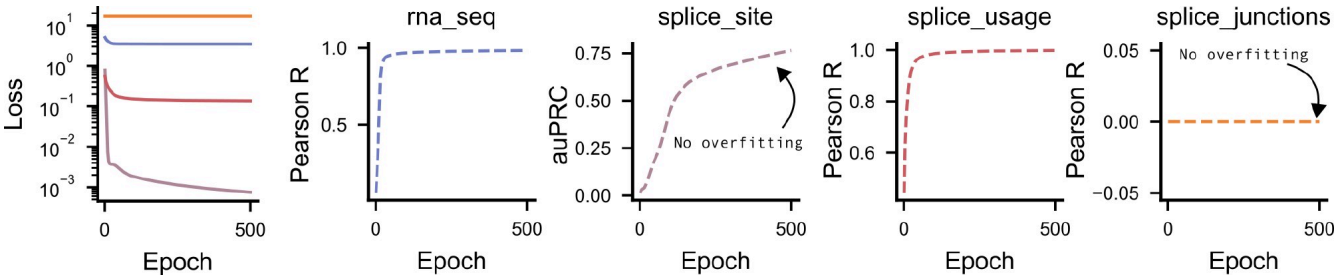


Normalizing training data on the fly.

First time never works

As a first sanity check, we verified that the model could overfit a single genomic interval using linear probing (freezing the backbone and training only the new heads). A model with enough capacity should memorize a single batch; failure to do so points to a bug rather than a generalization problem. To make the test representative, we selected an interval with median splice junction density among all training intervals (see this notebook for the selection details).

The test was useful precisely because it failed. Coverage and splice site usage overfit fine on the first attempt; the splice site classification head did not overfit, and the splice junction head did not learn at all. The next two sections trace what was wrong with each of those two heads and how we fixed it.



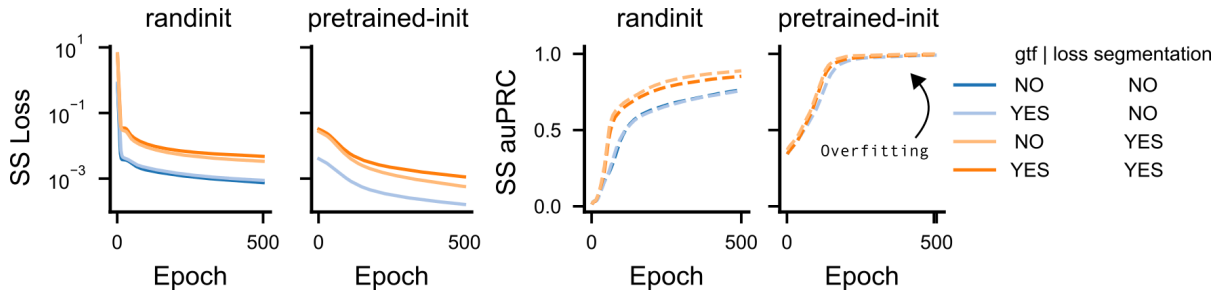
First attempt to overfit a single interval.

Why the splice site head could not overfit

The splice site classification head struggled to overfit when initialized randomly. This is not surprising: the purpose of fine-tuning AlphaGenome is not to re-learn the general splicing code from scratch. Unlike the splice site usage and junction heads, which are sample-specific (AlphaGenome exposes them per tissue/track), the splice site classification head captures a single, shared property across all samples: where donor and acceptor sites sit on each strand. There is one general splice site head, not a track to pick per sample or cell type, so it makes sense to start from its pretrained weights directly and let the sample-specific heads adapt instead.

We explored three factors that could affect overfitting on the single interval: (1) initializing the splice site classification head from pretrained rather than random weights, (2) augmenting the interval's splice sites with all annotated sites from the GTF (to reduce label sparsity, since AlphaGenome was pretrained on many more samples and thus more splice sites), and (3) segmenting the loss computation to match the 8-segment sequence parallelism used during AlphaGenome pretraining. We implemented support for all three options via `finetune.py` flags: `--pretrained-head-samples` to initialize specific head weights from a pretrained track (e.g. `splice_site:0`), `--gtf` to supply a GTF or parquet file of canonical splice sites, and `--num-segments` to control how many segments the sequence is split into for loss computation (applies to all modalities, not just splicing).

Initializing the splice site classification head from pretrained weights enabled clean overfitting regardless of whether GTF sites or loss segmentation were used. When starting from random weights, loss segmentation had a secondary effect: higher overall loss but faster overfitting, with or without the GTF augmentation.



Debugging the splice site classification head.

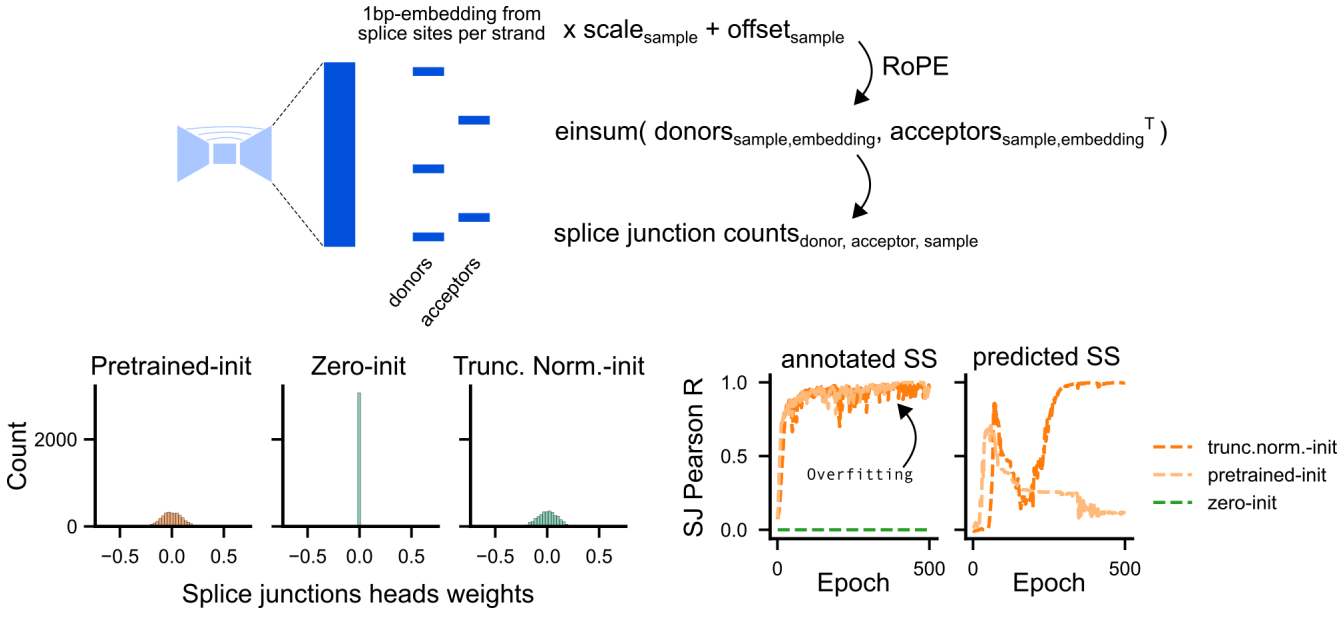
Why the junction head could not learn

In our first attempt, the splice junction head would not learn. Understanding why required looking at how the head works. For each genomic interval, the trunk produces 1 bp resolution embeddings across the full sequence. The junction head extracts embeddings only at the positions of known splice sites (up to 512; padded with -1 if fewer are present). Each extracted embedding is then linearly transformed with a learned per-sample scale and offset before RoPE positional encoding is applied. Junction counts for each donor-acceptor pair are then predicted as the softplus of the inner product between the corresponding donor and acceptor embeddings.

Inspecting the pretrained weights revealed the problem: the junction head's `rope_params`, which contain the learned scale and offset, were non-zero and approximately truncated-normal distributed, whereas a newly initialized head set both to zero. With `scale = 0` and `offset = 0`, the transformation `scale * x + offset` collapses every splice-site embedding to zero before RoPE is applied, preventing the head from learning properly. We confirmed the pretrained parameter distribution for predicting K562 polyA-plus RNA-seq stranded tracks, chosen only as an available stranded pretrained initialization rather.

Cross-referencing with the original JAX implementation and reaching out to the authors (see GitHub issue) confirmed this was a bug also in the original implementation. The fix was to initialize `rope_params` with a truncated normal distribution (std=0.1), controlled via the `--rope-init` flag in `finetune.py` (default: `truncated_normal`; `zeros` is kept only for ablation experiments).

With the fix in place, the randomly initialized junction head overfit the interval comparably to a head initialized from the pretrained K562 weights when splice-site positions were taken from the sample's own annotated, STAR-called junctions, using `--junction-position-source annotated` (the default). This position source is sample-dependent rather than GTF-derived: it uses the junctions observed in the sample to define the candidate splice-site positions for which the model learns to predict counts. We used the K562 weights only as a pretrained reference initialization, not because K562 is biologically matched to our samples.

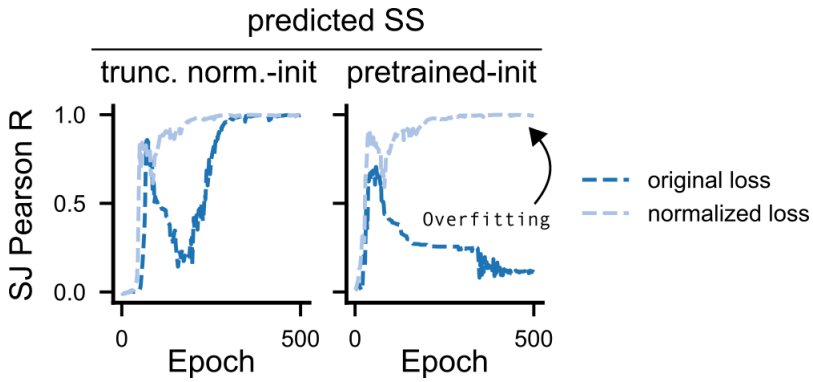


Debugging the splice junction head: random initialization.

Using predicted positions from the splicing head (`--junction-position-source predicted`) exposed a further issue: here it's the pretrained-weight initialization of the junction heads that collapsed, while random initialization still learned. A possible explanation is that the pretrained junction heads were optimized alongside a dense, tissue-diverse set of splice site positions from pretraining, suggesting that positions predicted by a freshly fine-tuned splice site head on just two samples are sparser and differently distributed, likely placing it outside its operating range. Looking at the loss values pointed to a related problem: the junction cross-entropy was going negative during training, which would destabilize any head but hit the pretrained one harder.

The root cause seemed to be a mismatch between the paper's pseudocode and both the JAX and PyTorch implementations. The supplementary methods define `multinomial_cross_entropy` by normalizing both targets and predictions to ratios before computing the cross-entropy, so `log(p_pred) <= 0` always and the loss is guaranteed non-negative. The JAX implementation instead uses a log-normalizer formulation (`log_normalizer - log_likelihood`), which is mathematically equivalent when junction counts are present but produces a negative contribution when a training window contains annotated splice sites with zero observed counts; common in sparse datasets like SF3B1 RNA-seq, where not every annotated site will have reads in a given sample. The PyTorch port (`--junction-loss original`, the default) inherited this behavior unchanged.

We opened another GitHub issue with the authors, who confirmed the discrepancy and introduced a ratio-normalized formulation: both targets and predictions are divided by their within-mask sums before computing `-p_true * log(p_pred)`. We ported this as `--junction-loss normalized`. Since `p_pred <= 1`, its log is always non-positive and the loss is always non-negative. With this correction, training is stable, the loss no longer dips below zero on our window, and both random and pretrained-weight initialization can overfit the single interval regardless of whether annotated or predicted splice site positions are used.



Debugging the splice junction head: loss formulation.

What actually worked: practical recipe for full fine-tuning AlphaGenome on new samples across RNA-seq modalities

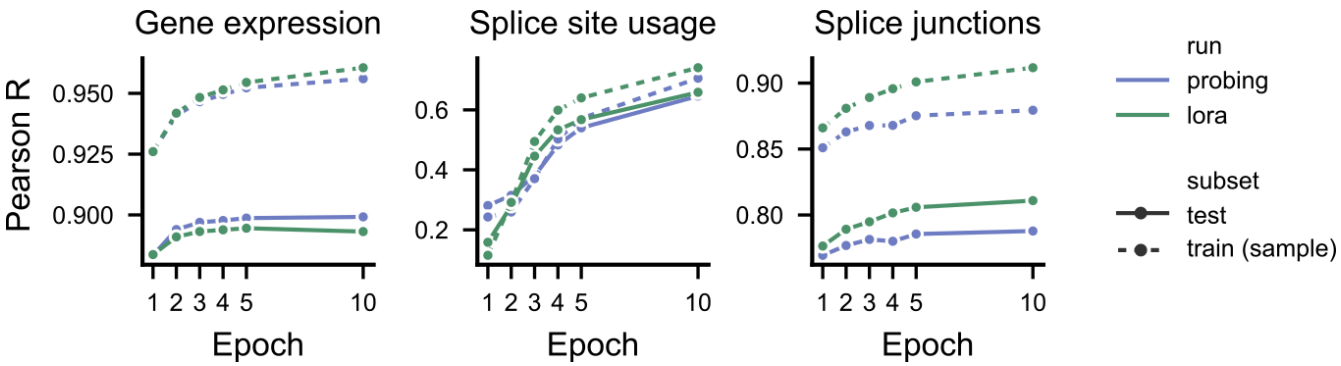
After resolving the failure points identified above, we scaled up to a first genome-wide fine-tuning run using linear probing, with the trunk frozen and only the new output heads trained. We used the settings that performed most reliably during single-interval debugging:

- pretrained initialization for the splice site head;
- annotated junction positions;
- truncated-normal initialization for the junction head's RoPE parameters;
- the normalized, ratio-based junction loss.

Using annotated junction positions gives the junction head a reliable training signal. Positions predicted by a splice site head that is still learning seemed to introduce an additional source of error. This follows AlphaGenome's pretraining strategy of first training on annotated positions and later distilling to predicted ones.

We also used eight-way loss segmentation. Although the single-interval experiments did not establish it as necessary, we retained it for consistency with AlphaGenome pretraining as well.

Performance on held-out intervals improved over training, alongside a modest but consistent generalization gap: at every epoch, we compared test-set performance against a same-sized random subsample of training intervals (`train_sample`). At epoch 10, held-out test performance trailed the training subsample by roughly 0.06–0.10 Pearson r across all three modalities (gene expression, splice site usage, splice junctions), for both linear-probe and LoRA. Reassuringly, that gap stayed roughly constant across epochs rather than widening — i.e. the models aren't progressively overfitting the training intervals with more training, though they also aren't closing the gap to training performance either.



Held-out (solid) vs. training-subsample (dashed) performance across full fine-tuning epochs, linear-probe vs. LoRA, for gene expression, splice site usage, and splice junctions.

Gene expression performance saturated early, so we asked whether LoRA fine-tuning - updating the trunk's internal DNA representation via low-rank adapters, rather than only the new output heads — could do better, particularly on the splicing-specific modalities. Comparing linear-probing against LoRA at epoch 10 on the held-out test set:

Metric	**Linear-probe**	**LoRA**	**Δ (LoRA − probe)**
Gene expression (Pearson r)	**0.899**	0.893	−0.006
Splice site usage (Pearson r)	0.646	**0.659**	+0.013
Splice junctions (Pearson r)	0.788	**0.811**	+0.023

As expected, LoRA, by updating the trunk's internal DNA representation instead of leaving it frozen, improved performance over linear probing on the splicing-specific metrics (splice site usage, splice junctions), while gene expression stayed essentially unchanged between the two (linear-probe even edges it out slightly there).

The two strategies also differ substantially in compute footprint, though less in runtime than in memory. Full genome fine-tuning took about a week on a single GPU for both strategies, since backpropagating through the frozen trunk to reach the low-rank adapters in LoRA still requires storing its activations end to end, even though the trunk's own weights aren't updated. Where the two diverge is memory: LoRA needed a full NVIDIA H100 (80 GB), while linear

probing, which only needs gradients for the new heads, ran comfortably on NVIDIA Hopper-class GPUs with 64 GB. See Xinming's GPU benchmarking post for a closer look at how these memory/compute tradeoffs play out across fine-tuning strategies.

The workflows for full fine-tuning and evaluation are available in the repository: `workflows/05-full_finetuning` and `workflows/06-evaluation`.

Limitations

The initial debugging (up to and including the loss-formulation fix) was performed on a single genomic interval with just two RNA-seq samples, so those specific observations may not generalize to larger or more diverse training sets — this is why we followed up with the full genome-wide fine-tuning run above. The preprocessing pipeline reproduces the main steps described in the AlphaGenome paper, but since the original code is not public, we cannot guarantee exact parity.

The splicing fine-tuning code currently lives in a dedicated fork of alphagenome-pytorch and is pending merge into the main branch. Support in the JAX-based alphagenome_ft will follow once that merge is complete. We'll update this post once both merges land, with any code or figure references that change as a result.

This post focuses on getting the implementation correct and measuring held-out performance, not on biological interpretation. We do not yet relate these predictions to independent data or ask whether the fine-tuned heads capture SF3B1-specific splicing changes — that comparison, and the accompanying biological interpretation, is left for the follow-up post. We also only evaluate two RNA-seq samples (one WT, one K700E) and a single AlphaGenome fold; generalization to more samples, mutations, or cell types remains to be tested.

Conclusion

This process gave us a much better understanding of what it actually takes to fine-tune new transcriptomic heads on AlphaGenome. Extending the model to splicing modalities was not simply a matter of adding data loaders and output layers (as we hoped): higher label sparsity, head initialization, loss formulation, and the interaction between position sources and weight initialization all turned out to matter.

The most useful strategy was to build confidence at each step before scaling up, from preprocessing validation and target inspection to single-interval overfitting. Each stage exposed a different class of problem. The single-interval test in particular was worth its weight as the two loss bugs and a zero-initialization bug in the junction head would have been much harder to diagnose in a full training run. We hope that sharing these intermediate failures, alongside the fixes and the flags that control them, makes it easier for others to extend AlphaGenome to new RNA-seq modalities and biological questions.

The full fine-tuning results also suggest that gene expression and splicing tasks don't need the same amount of compute: gene expression performance saturated early and didn't benefit from LoRA, so linear probing on modest hardware is likely enough there, whereas the splicing modalities kept improving and gained the most from LoRA's larger compute and memory footprint. If gene expression is your main target, linear probing is probably all you need; splicing tasks are where the extra compute marginally pays off.

In the end, this project became more than a simple port. It resulted in reusable preprocessing scripts, training pipelines, and a set of practical checks and warnings that we hope make fine-tuning RNA-seq–derived splicing modalities on AlphaGenome more transparent and reproducible. We are especially grateful to the DeepMind

developers for openly sharing their code and model weights, and for the responsiveness of Tom Ward (`@tomwardio`) and Vincent Dutordoir (`@vdutor`) when we reported bugs and implementation issues; their feedback was instrumental in reaching the conclusions presented here. We hope these efforts help make fine-tuning splicing heads as seamless and accessible as possible for the broader community.

Reproducibility

The repository alphagenome_finetuning_rna contains all the necessary code, from data downloading to analysis and figures, to reproduce these results.

References

1. Avsec, Ž. et al. Advancing regulatory variant effect prediction with AlphaGenome., 649, Nature (2026). https://doi.org/10.1038/s41586-025-10014-0
2. López-Oreja, I. et al. SF3B1 mutation-mediated sensitization to H3B-8800 splicing inhibitor in chronic lymphocytic leukemia., 6, Life Sciences Alliance (2023). https://doi.org/10.26508/lsa.202301955
3. Dent, C.I. et al. Quantifying splice-site usage: a simple yet powerful approach to analyze splicing., 3, NAR Genomics and Bioinformatics (2021). https://doi.org/10.1093/nargab/lqab041
4. Ramírez, F. et al. deepTools2: a next generation web server for deep-sequencing data analysis., 44, Nucleic Acids Research (2016). https://doi.org/10.1093/nar/gkw257
5. Dobin, A. et al. STAR: ultrafast universal RNA-seq aligner., 29, Bioinformatics (2013). https://doi.org/10.1093/bioinformatics/bts635

Acknowledgements

Thanks to the Genomics x AI community and the Kundaje Lab at Stanford, where the AlphaGenome PyTorch port is being developed as well. We acknowledge the EuroHPC Joint Undertaking for awarding this project access to the EuroHPC supercomputer MareNostrum 5, hosted by the Barcelona Supercomputing Center (Spain) through an EuroHPC Development Access call. We acknowledge support of the Spanish Ministry of Science and Innovation through the Centro de Excelencia Severo Ochoa (CEX2020-001049-S, MCIN/AEI /10.13039/501100011033), and the Generalitat de Catalunya through the CERCA programme, and to the EMBL partnership. We are grateful to the CRG Core Technologies Programme for their support and assistance in this work.